



DAPPW - Despliegue de aplicaciones web

Diego Ayala

Resultados de aprendizaje - RA

- RA1- Implanta arquitecturas web analizando y aplicando criterios de funcionalidad.
- RA2- Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
- RA3- Implanta aplicaciones web en servidores de aplicaciones, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
- RA4-Administra servidores de transferencia de archivos, evaluando y aplicando criterios de configuración que garanticen la disponibilidad del servicio.

Resultados de aprendizaje - RA

- RA5-Verifica la ejecución de aplicaciones web comprobando los parámetros de configuración de servicios de red.
- RA6-Elabora la documentación de la aplicación web evaluando y seleccionando herramientas de generación de documentación, control de versiones y de integración continua.

RA- Equivalencia con el temario

- RA1- Arquitecturas web, Docker y Docker Compose
- RA2- Nginx: proxy inverso, estáticos, multimedia y HTTPS.
- RA3- Gunicorn + Django: configuración y despliegue.
- RA4- Transferencia de archivos (compartido con FEM2).
- RA5- DNS y servicios de directorio.
- RA6- Documentación, Git e integración continua.

RA- Evaluación y ponderación

● RA1- Práctica1 (memoria y defensa oral)	20%
● RA2- Práctica1 (memoria y defensa oral)	25%
● RA3- Práctica1 (memoria y defensa oral)	15%
● RA4- Compartido con FEM2 (según Plan de Formación).	10%
● RA5- Práctica2 (memoria y/o defensa oral)	10%
● RA6- Práctica1 (memoria y defensa oral)	20%
	100%

Práctica1: Compose Dev + Prod 1 + Prod 2 + ampliación.

TODOS LOS RA DEBERÁN SER APROBADOS

Evaluación final y recuperación de RA

- Evaluación final ordinaria — 1.ª convocatoria del curso
Se tendrán en cuenta las evidencias obtenidas durante el curso. Las actividades de recuperación se centrarán en los RA pendientes. Si todos los RA están superados, no será necesario realizar pruebas de recuperación.
- Periodo de recuperación
El alumnado con RA pendientes seguirá un programa con clases, resolución de dudas, actividades, trabajos o pruebas dirigidas a su superación.
- Evaluación final extraordinaria — 2.ª convocatoria del curso
Se evaluarán únicamente los RA que continúen pendientes. Los RA ya superados conservarán su calificación. Se aplicarán los mismos criterios de calificación y ponderaciones establecidos en la programación.

También podrán recuperarse RA durante el curso mediante nuevas evidencias.

Cuando no pueda aplicarse la evaluación continua

● Asistencia

Con carácter general, al alcanzar el **30 % de faltas justificadas e injustificadas** sobre las horas lectivas de la fase de centro del módulo, se aplicará el procedimiento de evaluación diferenciada.

● Qué se evaluará

Se comprobarán los RA y CE que no hayan quedado suficientemente acreditados. Las evidencias anteriores válidas podrán conservarse: **no se considerarán automáticamente suspensos todos los RA.**

● Actividades y pruebas

Podrán utilizarse las actividades y pruebas previstas o sustituirlas por otras equivalentes. Se mantendrán los mismos RA, CE, ponderaciones y criterios de superación.

● Información previa

Cuando se utilicen actividades o pruebas diferentes de las inicialmente previstas, sus características y condiciones se comunicarán con una **antelación mínima de una semana.**

Formación en empresa (FEM1) y evaluación

- **Lenguajes de Marcas · Modelo concentrado**

- **Calificación de la fase de centro**

La nota obtenida en primero se utilizará para decidir la promoción a segundo. En este módulo, la calificación quedará pendiente de cierre hasta completar y superar la FEM1.

- **Resultados de aprendizaje compartidos**

La empresa desarrollará y valorará los RA establecidos en el Plan de Formación individual, de forma compartida con el centro. No todos los RA del módulo se evalúan en la empresa.

Formación en empresa (FEM1) y evaluación

● **Cómo influye la valoración de la empresa**

Según el procedimiento establecido en la programación:

- *Superado*: se valida la parte desarrollada en empresa y se mantiene la calificación del RA obtenida en el centro.
- *No superado*: el RA queda pendiente y deberá recuperarse.
- *No superado-no demostrado*: si no pudo demostrarse por causas ajenas al alumnado, no habrá una penalización automática y se facilitará una evidencia alternativa.

● **Calificación definitiva del módulo**

La determina el profesor responsable, incorporando el informe de empresa y aplicando los criterios de la programación. La empresa aporta su valoración; no sustituye la nota del centro por una nota numérica propia.

Entregas e incidencias

● Entrega de actividades

Las actividades se entregarán mediante la tarea de Microsoft Teams o el canal indicado, respetando la fecha, la hora y el formato establecidos en el enunciado.

● Archivos completos y accesibles

Se entregará el código fuente y los archivos solicitados. Antes de enviar, deberá comprobarse que se pueden abrir, que las rutas funcionan y que los enlaces tienen los permisos adecuados. Las capturas no sustituirán al código cuando este sea requerido.

● Problemas técnicos

Cualquier incidencia que impida entregar o realizar una prueba deberá comunicarse al profesor tan pronto como sea posible, aportando la información disponible. Se valorará la situación y se indicará cómo proceder conforme a la programación.

● Ausencia a una prueba o defensa

La ausencia deberá comunicarse y justificarse por los canales del centro. Cuando proceda una actividad alternativa, su fecha y condiciones serán establecidas por el profesor conforme a la programación.

Autoría, trabajo individual y uso de IA

● **Autoría y recursos externos**

No se presentarán trabajos ajenos como propios. Cuando se utilicen fragmentos de código, imágenes u otros materiales externos permitidos, se identificará su procedencia y la aportación realizada.

● **Comprobación individual del aprendizaje**

El profesor podrá solicitar una explicación, defensa o modificación del código entregado. En los trabajos grupales, cada integrante deberá acreditar su aprendizaje y su contribución.

● **Uso de inteligencia artificial**

En cada actividad se indicará qué uso de IA está permitido. Cuando se utilice, se identificará la herramienta y para qué se ha empleado. El alumnado deberá revisar el resultado y poder explicar las soluciones presentadas.

● **Seguridad y confidencialidad**

En las prácticas se utilizarán preferentemente datos ficticios. No se publicarán contraseñas, claves de API, datos personales de terceros ni información confidencial en repositorios, entregas públicas o herramientas de IA.

Asistencia: evaluación continua y matrícula

- **Evaluación continua del módulo**

La acumulación de faltas puede impedir aplicar la evaluación continua. En ese caso se seguirá el procedimiento de evaluación diferenciada previsto en la programación.

- **Permanencia en el ciclo**

La falta de incorporación o la inasistencia reiterada también pueden dar lugar a un procedimiento de anulación de matrícula.

- **Son procedimientos diferentes**

Perder la evaluación continua de un módulo no supone automáticamente perder la matrícula, ni considerar suspensos todos los RA previamente acreditados.

Inasistencia y posible anulación de matrícula

- **Dirección podrá iniciar el procedimiento cuando se produzca alguno de estos supuestos:**

- Al comienzo del curso**

- No asistir durante los cinco primeros días lectivos.



- Durante el curso**

- Acumular diez días lectivos consecutivos sin asistir y sin justificación.



- Acumulación general**

- Superar el 30 % de **faltas injustificadas** de los periodos lectivos obligatorios computables, excluidos los módulos convalidados, renunciados y pendientes de cursos anteriores.

La **anulación de matrícula** no es automática: requiere su correspondiente procedimiento.

Referencia: Decreto 13/2026, artículo 26.4.a.

Comunicaciones y derecho a presentar alegaciones

- **Revisa las comunicaciones del centro**

Mantén actualizados tus datos de contacto y consulta los canales oficiales indicados.

- **Comunica cualquier error o incidencia**

Si una falta está mal registrada o dispones de documentación justificativa, comunícalo cuanto antes al tutor.

- **Procedimiento de anulación de matrícula**

Dirección comunicará su inicio y concederá tres días hábiles para presentar alegaciones y documentación. Después resolverá en un máximo de cinco días hábiles y comunicará la decisión.

La resolución podrá recurrirse por los cauces indicados en su notificación.

Referencia del procedimiento: Orden de evaluación, artículo 13. Las dos primeras indicaciones son pautas organizativas propuestas.

Justificación de ausencias y puntualidad

- **Comunica y justifica las ausencias**

Utiliza el procedimiento del centro (notificarlo por correo al tutor) y aporta la documentación que corresponda. Si eres menor de edad, intervendrán tus representantes legales.

- **Justificar no equivale a haber asistido**

La justificación permite distinguir la causa de la ausencia, pero no sustituye las actividades de aprendizaje ni las evidencias pendientes.

- **Incidencias prolongadas**

Ante una enfermedad, accidente, dificultad familiar o incompatibilidad sobrevenida, contacta cuanto antes con tutoría y Jefatura para valorar cómo continuar tu formación.

- **Retrasos y salidas anticipadas**

Se registrarán y tratarán conforme a las normas del centro.

Becas: una obligación de asistencia diferente

- **La beca tiene sus propias condiciones**

Las reglas para conservarla no son las mismas que las de evaluación continua o matrícula.

- **Asistencia mínima**

Con carácter general, debe asistirse al 80 % o más de las horas lectivas, salvo las excepciones aplicables.

- **Posible devolución**

La falta de asistencia exigida o la baja durante el curso pueden originar la devolución de la beca.

Asistencia durante la FEM

- **Cumplimiento del horario**

Durante la FEM deben respetarse el calendario, el horario y las condiciones del plan de formación.

- **Comunicación de incidencias**

Las ausencias y retrasos se comunicarán al tutor de empresa y al tutor del centro por los canales acordados.

- **Faltas reiteradas**

Las faltas repetidas de asistencia o puntualidad sin justificar pueden comprometer la continuidad de la estancia.

- **No se aplica un “permiso para faltar el 30 %”**

Las ausencias y su posible recuperación o reorganización se resolverán conforme al plan y al convenio de formación.

El modelo de convenio del anexo XVI del RD 659/2023, cláusula séptima, contempla la rescisión de la participación por faltas repetidas de asistencia o puntualidad no justificadas.

Huelga e inasistencia colectiva

- **Decisión colectiva y comunicación previa**

Debe acordarse en el ejercicio del derecho de reunión y comunicarse previamente a Dirección por el cauce del centro.

- **No es una falta de conducta**

Desde 3.º de ESO, si se cumplen esas condiciones, la inasistencia colectiva no puede sancionarse como falta de conducta. Esta protección está recogida en el artículo 8 de la LODE y en el artículo 23.4 del Decreto murciano de convivencia.

- **Respeto a todas las decisiones**

Debe respetarse tanto a quien participe como a quien decida asistir a clase.

Consulta la organización del centro

Consulta con tutoría o Jefatura el trámite de comunicación y la organización de clases, pruebas y entregas.

- **Una convocatoria en redes no sustituye la comunicación al centro. Se tratará como falta justificada, afectado al 30% de pérdida de ev continua pero no al 30% de expulsión del centro**

Convocatorias de evaluación

- **Dos convocatorias por curso**
Evaluación final ordinaria y evaluación final extraordinaria.
- **Máximo general de cuatro por módulo**
Se cuentan a lo largo de los cursos; no son cuatro nuevas cada año.
- **Parciales y recuperaciones**
Los controles, prácticas y recuperaciones internas no son convocatorias finales adicionales.
- **Convocatoria extraordinaria de gracia**
Si se agotan, puede solicitarse por causas justificadas, sin superar la permanencia máxima. Requiere autorización motivada; no es automática.
- **Si no puedes seguir o presentarte, consulta la renuncia: no basta con faltar.**



Renuncia a una convocatoria

- **Cuándo puede solicitarse**

Por enfermedad prolongada o accidente, trabajo, obligaciones personales o familiares, maternidad/paternidad u otras causas excepcionales justificadas.

- **Plazos de solicitud**

Para la evaluación final ordinaria, al menos un mes antes. Para la extraordinaria, dentro de los tres días siguientes a la comunicación de la calificación ordinaria.

- **Efectos de la renuncia concedida**

No consume esa convocatoria, pero no detiene el plazo de permanencia en el ciclo.

- **Avisar al profesor o faltar a la prueba no equivale a una renuncia concedida.**

- **La renuncia y la anulación de matrícula son trámites diferentes; para más concreción a Jefatura**

Calendario escolar



CALENDARIO ESCOLAR 2026-2027
(PUBLICADO BORM 11-06-2026)

CARTAGENA

SEPTIEMBRE						
L	M	X	J	V	S	D
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

OCTUBRE							NOVIEMBRE							DICIEMBRE						
L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D
			1	2	3	4							1	1	2	3	4	5	6	
5	6	7	8	9	10	11	2	3	4	5	6	7	8	7	8	9	10	11	12	13
12	13	14	15	16	17	18	9	10	11	12	13	14	15	14	15	16	17	18	19	20
19	20	21	22	23	24	25	16	17	18	19	20	21	22	21	22	23	24	25	26	27
26	27	28	29	30	31		23	24	25	26	27	28	29	28	29	30	31			
							30													
ENERO							FEBRERO							MARZO						
L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D
				1	2	3	1	2	3	4	5	6	7	1	2	3	4	5	6	7
4	5	6	7	8	9	10	8	9	10	11	12	13	14	8	9	10	11	12	13	14
11	12	13	14	15	16	17	15	16	17	18	19	20	21	15	16	17	18	19	20	21
18	19	20	21	22	23	24	22	23	24	25	26	27	28	22	23	24	25	26	27	28
25	26	27	28	29	30	31								29	30	31				
ABRIL							MAYO							JUNIO						
L	M	X	J	V	S	D	L	M	X	J	V	S	D	L	M	X	J	V	S	D
			1	2	3	4						1	2	1	2	3	4	5	6	
5	6	7	8	9	10	11	3	4	5	6	7	8	9	7	8	9	10	11	12	13
12	13	14	15	16	17	18	10	11	12	13	14	15	16	14	15	16	17	18	19	20
19	20	21	22	23	24	25	17	18	19	20	21	22	23	21	22	23	24	25	26	27
26	27	28	29	30			24	25	26	27	28	29	30	28	29	30				
							31													

ED. INFANTIL/PRIMARIA/ESPECIAL-	Inicio, fin y día del patrón.
ESO/BACHILLERATO/FP BÁSICA-	Inicio, fin y día del patrón.
FP DE GRADO MEDIO Y SUPERIOR.-	Inicio, fin y día del patrón.
Días FESTIVOS	
Días NO LECTIVOS	

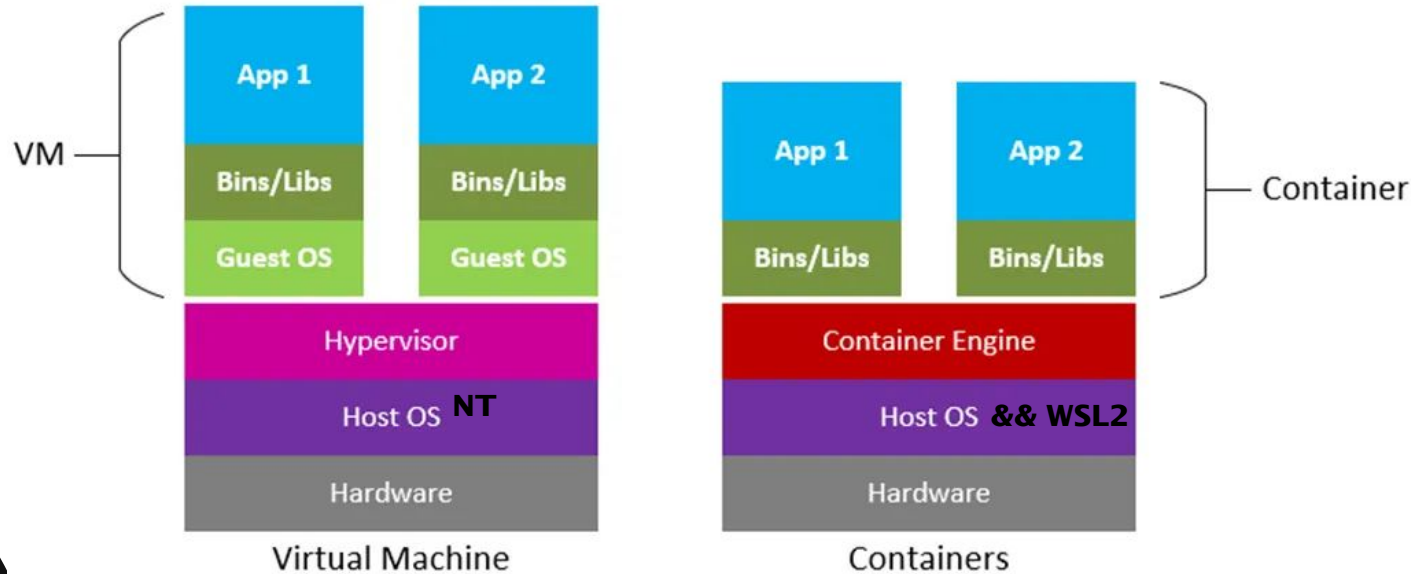


DAPPW - 0.Virtualización

Diego Ayala

MVs vs Docker

- Ambos sirven para aislar aplicaciones del sistema.
- Impacto en rendimiento, recursos y velocidad.
- Hypervisor → simula hardware para cada VM.
- Kernel → Gestiona CPU, memoria, procesos, disco, red → Dentro del OS





DAPPW - 1.Docker

Diego Ayala

Docker- Docker Desktop

- [Docker Desktop](#)
- Docker desktop necesita WSL2 → proporciona el kernel de linux

Docker Desktop for Windows - x86_64

Docker Desktop for Windows - x86_64 on the Microsoft Store

Docker Desktop for Windows - Arm (Early Access)

Docker- WSL2

- CMD en modo **administrador**
- Reiniciamos el SSOO para aplicar los cambios

```
C:\Windows\System32>wsl --version
El Subsistema de Windows para Linux no está instalado. Para instalar, ejecute "wsl.exe --install".
Para obtener más información, visite https://aka.ms/wslinstall

Presione cualquier tecla para instalar Subsistema de Windows para Linux.
Presione CTRL-C o cierre esta ventana para cancelar.
Esta solicitud agotará el tiempo de espera en 60 segundos.
```

```
Descargando: Subsistema de Windows para Linux 2.5.9
[=====49,0%
```

Docker- WSL2

```
Descargando: Subsistema de Windows para Linux 2.5.9
Instalando: Subsistema de Windows para Linux 2.5.9
Se ha instalado Subsistema de Windows para Linux 2.5.9.
Instalando componente opcional de Windows: VirtualMachinePlatform

Herramienta Administración y mantenimiento de imágenes de implementación
Versión: 10.0.26100.1150

Versión de imagen: 10.0.26100.4349

Habilitando características
[=====100.0%=====]
La operación se completó correctamente.
La operación solicitada se realizó correctamente. Los cambios se aplicarán una vez que se reinicie el sistema.
Versión de WSL: 2.5.9.0
Versión de kernel: 6.6.87.2-1
Versión de WSLg: 1.0.66
Versión de MSRDC: 1.2.6074
Versión de Direct3D: 1.611.1-81528511
Versión de DXCore: 10.0.26100.1-240331-1435.ge-release
Versión de Windows: 10.0.26100.4349
```

Docker- WSL2

- Comprobamos que se ha instalado correctamente

```
C:\Windows\System32>wsl --version
Versión de WSL: 2.5.9.0
Versión de kernel: 6.6.87.2-1
Versión de WSLg: 1.0.66
Versión de MSRDC: 1.2.6074
Versión de Direct3D: 1.611.1-81528511
Versión de DXCore: 10.0.26100.1-240331-1435.ge-release
Versión de Windows: 10.0.26100.4349
```

Docker- WSL2

- Procedemos a la instalación de docker desktop

Installing Docker Desktop 4.42.1 (196648)

Configuration

☒ Add shortcut to desktop

OK



Welcome to Docker Desktop

Sign in to connect to your Docker Desktop subscription or access online features.

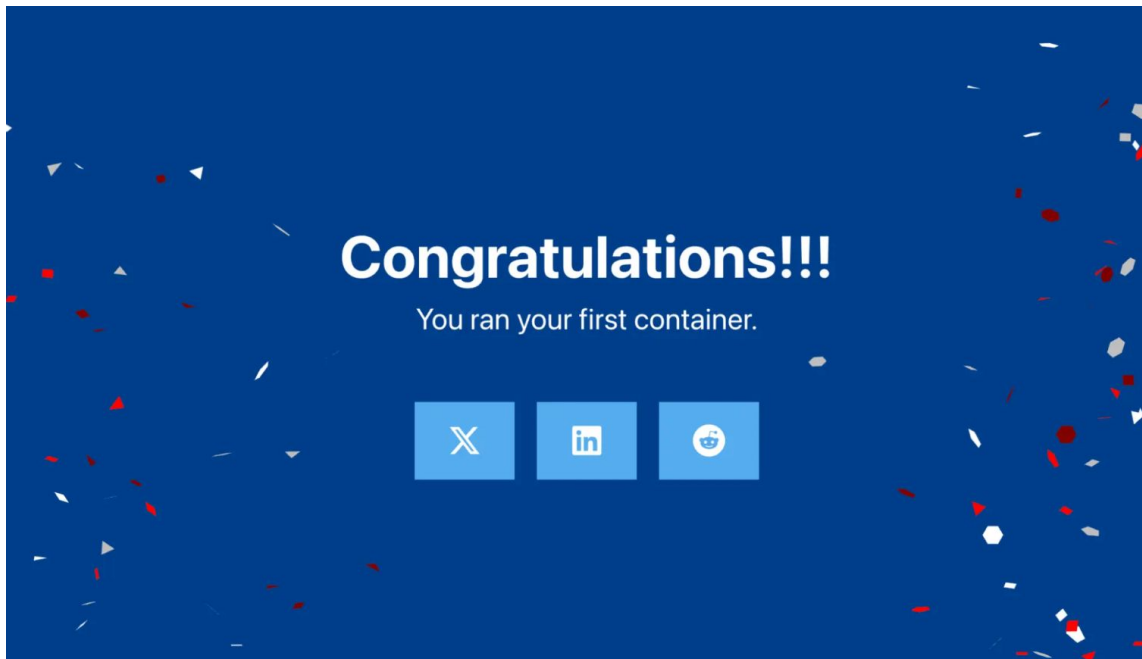
Sign up

Already have an account? [Sign in](#)

[Continue without signing in](#)

Ejecutando mi primer contenedor

- `docker run -d -p 8080:80 docker/welcome-to-docker`



Desglose gráfico del comando Docker

Ejecutando mi primer contenedor

```
>_ docker run -d -p 8080:80 docker/welcome-to-docker
```

docker run

Crea y arranca
un contenedor

-d

Ejecuta el contenedor
en segundo plano

-p 8080:80

Asocia puertos:
8080 en tu PC → 80
en el contenedor

docker/welcome-to-docker

Imagen usada para crear
el contenedor

¿Cómo funcionan los puertos?



Navegador

localhost:8080

Accedes desde tu navegador
a tu PC en el puerto 8080



Contenedor

puerto 80

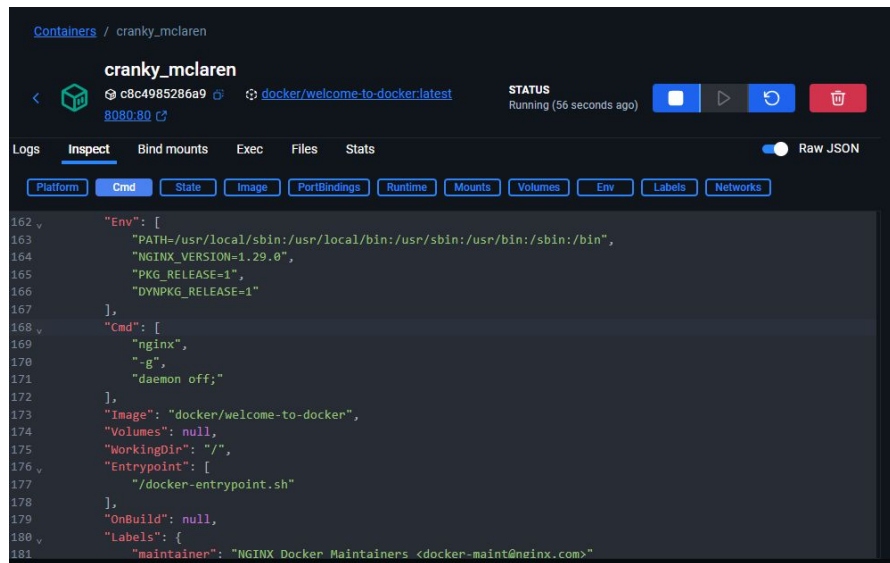
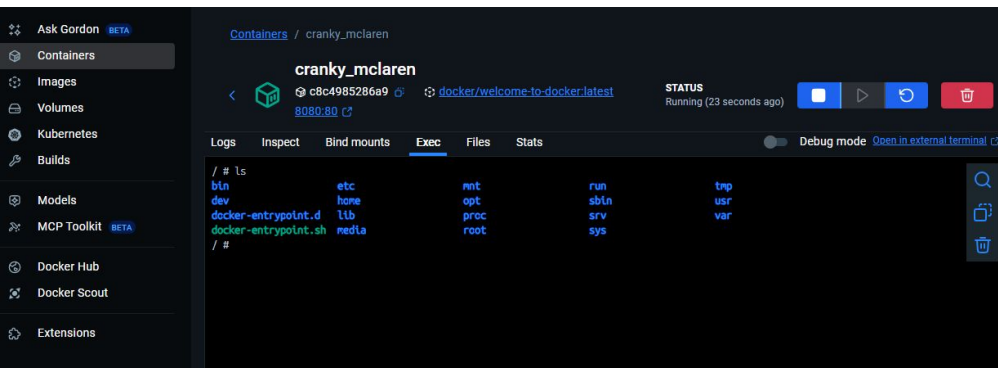
La aplicación dentro del
contenedor escucha en
el puerto 80



En resumen: Docker crea un contenedor a partir de una imagen, lo deja funcionando en segundo plano y publica su web en <http://localhost:8080>

Administrar contenedores mediante Docker Desktop

- Podemos acceder al shell mediante comandos.
- Mostrar información sobre el contenedor



Docker

- `docker compose watch` → vigila archivos de tu proyecto y, al cambiar algo, sincroniza el contenedor y/o reconstruye y reinicia el servicio.

```
18 FROM base AS client-base
19 COPY client/package.json client/package-lock.json ./
20 RUN npm install
21 COPY client/.eslintrc.cjs client/index.html client/vite.config.js ./
22 COPY client/public ./public
23 COPY client/src ./src
24
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\diego\Desktop\dapw\Docker\getting-started-todo-app-main\getting-started-todo-app-main> `docker compose watch`

Saliendo de la prehistoria

- cmd → pws 7
- winget search Microsoft.PowerShell
- winget install --id Microsoft.PowerShell --source winget

```
18 FROM base AS client-base
19 COPY client/package.json client/package-lock.json ./
20 RUN npm install
21 COPY client/.eslintrc.cjs client/index.html client/vite.config.js ./
22 COPY client/public ./public
23 COPY client/src ./src
24
```

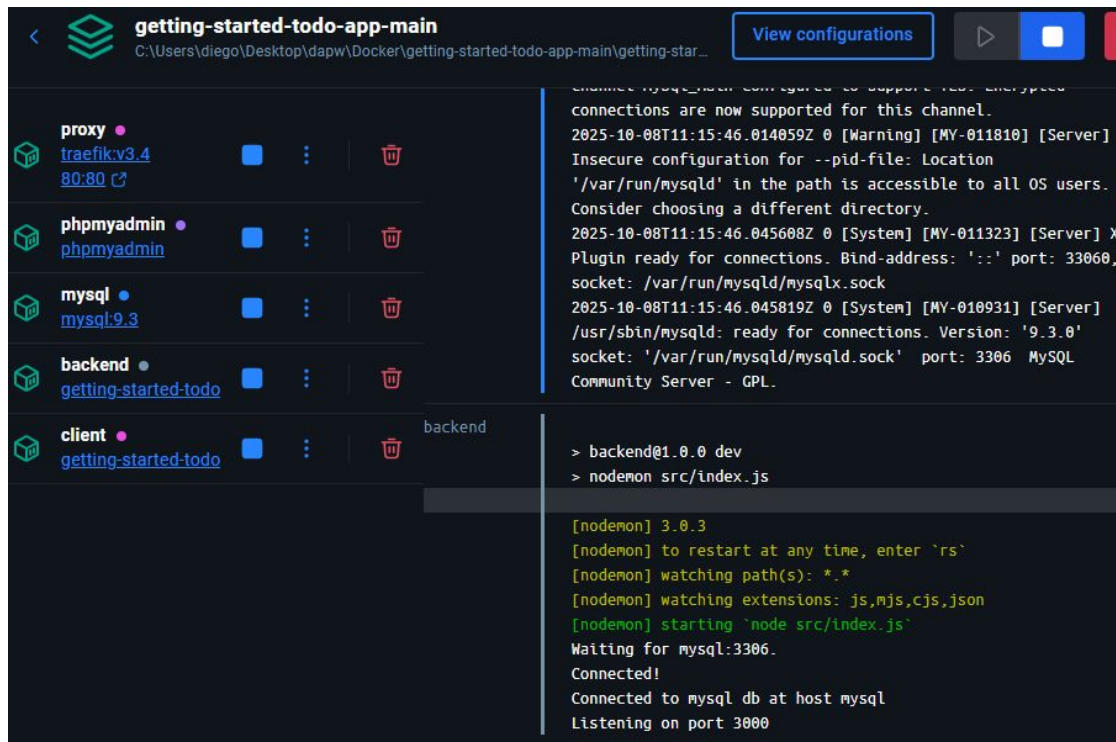
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

pwsh

C:\Users\diego\Desktop\dapw\docker\getting-started-todo-app-main\getting-started-todo-app-main> docker compose watch

Docker

- docker compose watch → contenedores desplegados



getting-started-todo-app-main
C:\Users\diego\Desktop\dapw\Docker\getting-started-todo-app-main\getting-star...

proxy ●
traefik:v3.4
80:80 ↗

phpmyadmin ●
phpmyadmin

mysql ●
mysql:9.3

backend ●
getting-started-todo

client ●
getting-started-todo

backend

terminal mysql:9.3.0 ready for connections. Version: '9.3.0' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.

> backend@1.0.0 dev
> nodemon src/index.js

[nodemon] 3.0.3
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node src/index.js'
Waiting for mysql:3306.
Connected!
Connected to mysql db at host mysql
Listening on port 3000

Docker - Ejercicio 1

- En backend/src/routes/[getGreeting.js](#). Modifica la variable greetings para que contenga un array de saludos que se mostrará de manera randomizada al usuario

Deberás hacer uso de las funciones [Math.floor\(\)](#) y [Math.random\(\)](#)

no os fieis de la comida de joaquin

Add Item

No items yet! Add one above!

Docker - Ejercicio 2

- En `client/src/components/AddNewItemForm.jsx` modifica el placeholder del label para cambiar el mensaje predeterminado por 'Que necesitas zangano?'

no os fieis de la comida de joaquin

Add Item

No items yet! Add one above!

Docker - Ejercicio 3

- En `client/src/index.scss` ajusta el atributo `background-color` al color que prefieras.

no os fieis de la comida de joaquin

Add Item

No items yet! Add one above!

Docker images, docker hub

- Imágenes → paquetes ligeros e independientes que contienen el código, el entorno de ejecución, las bibliotecas, las variables de entorno y los archivos de configuración necesarios para ejecutar un software.
- Docker Hub → plataforma que aloja imágenes de contenedores

The screenshot shows the Docker Hub interface for the user 'diegoayaladocker'. The top navigation bar is blue with the Docker Hub logo, 'Explore', 'My Hub', a search bar, and user controls. The left sidebar shows the user's profile and navigation links. The main content area displays the 'Repositories' section for the user's namespace.

Repositories
All repositories within the **diegoayaladocker** namespace.

Search by repository name: All content:

Name	Last Pushed ↑	Contains	Visibility	Scout
diegoayaladocker/getting-started-to...	14 days ago	IMAGE	Public	Inactive

1-1 of 1

Docker hub

The screenshot displays the Docker Hub web interface. At the top is a blue navigation bar with the Docker Hub logo, 'Explore', 'My Hub' (selected), and a search bar. On the left is a sidebar for the user 'diegoayaladocker' (Docker Personal), containing links to Repositories, Hardened Images (marked NEW), Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area shows the repository 'diegoayaladocker/test-daw' with a 'General' tab selected. It includes a breadcrumb trail, a 'Public view' button, Docker commands for pushing a tag, and a 'Tags' section with an 'INCOMPLETE' status and a 'DOCKER SCOUT INACTIVE' warning. A right-hand sidebar promotes 'buildcloud' with text about accelerating build times and a 'Go to Docker Build Cloud' link.

hub Explore My Hub Search Docker Hub CtrlK ? 🔔 🌙 ☰ D

diegoayaladocker Docker Personal

Repositories

Hardened Images NEW

Collaborations

Settings

Default privacy

Notifications

Billing

Usage

Pulls

Storage

Repositories / test-daw / General

diegoayaladocker/test-daw

Created less than a minute ago

todo-app

Add a category

Using 0 of 1 private repositories.

Docker commands

Public view

To push a new tag to this repository:

```
docker push diegoayaladocker/test-daw:tagname
```

General Tags Image Management BETA Collaborators Webhooks Settings

Tags INCOMPLETE

DOCKER SCOUT INACTIVE Activate

Pushed images appear here.

buildcloud

Build with Docker Build Cloud

Accelerate image build times with access to cloud-based builders and shared cache.

Docker Build Cloud executes builds on optimally-dimensioned cloud infrastructure with dedicated per-organization isolation.

Get faster builds through shared caching across your team, native multi-platform support, and encrypted data transfer - all without managing infrastructure.

Go to Docker Build Cloud →

Docker: Extensiones V

- Docker hub
- Dev Container

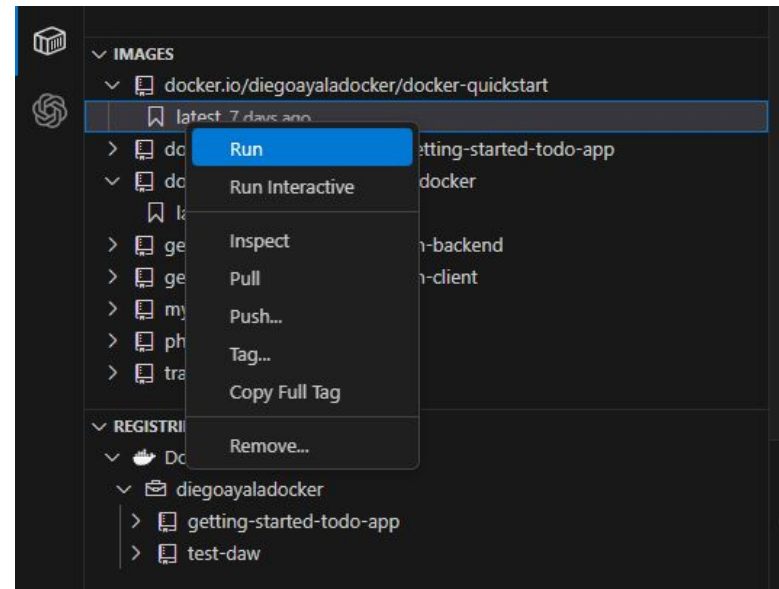
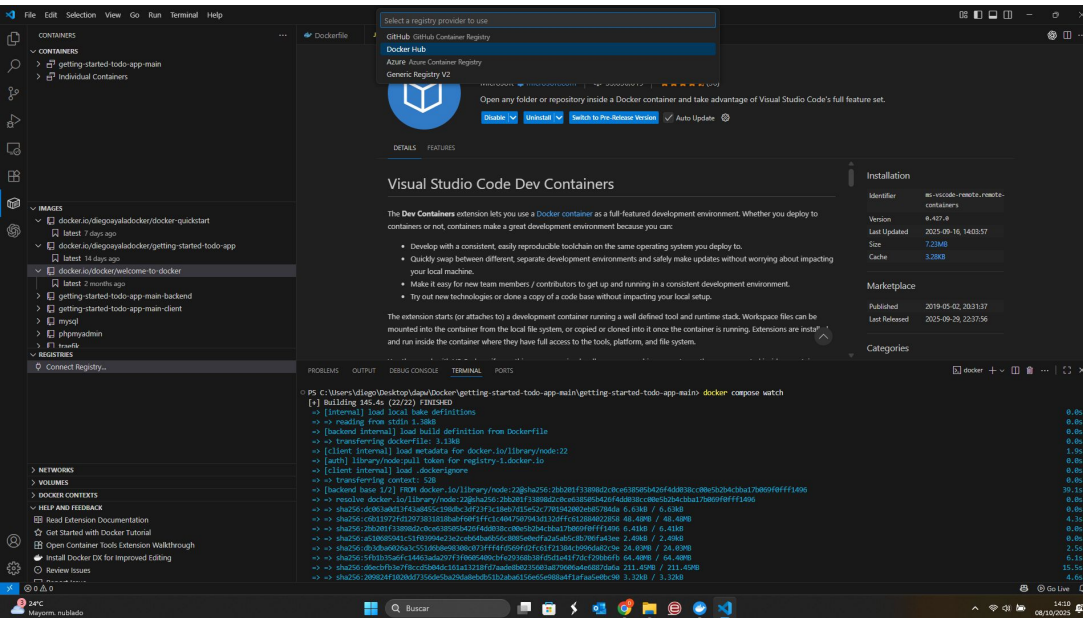
The screenshot shows the Visual Studio Code interface with the Docker extension installed. The left sidebar displays the 'CONTAINERS' view with a list of containers and images. The 'IMAGES' view shows a list of Docker images, including 'docker.io/diegoayaladocker/docker-quickstart' and 'docker.io/diegoayaladocker/getting-started-todo-app'. The 'REGISTRIES' view shows a list of registries, including 'docker.io' and 'myregistry.io'. The 'Dev Containers' extension is also visible, with a 'Dev Container' button and a 'Disable' button. The main editor area shows the 'Dockerfile' for the 'getting-started-todo-app' container, which includes instructions for building the image and running the container. The terminal at the bottom shows the output of the Docker commands, including the build and run processes.

The screenshot shows the Docker extension context menu in Visual Studio Code. The menu is open, displaying options for the selected image 'docker.io/diegoayaladocker/docker-quickstart'. The options include 'Run', 'Run Interactive', 'Inspect', 'Pull', 'Push...', 'Tag...', 'Copy Full Tag', and 'Remove...'. The 'Run' option is highlighted in blue. The background shows the Docker extension interface with a list of images and a search bar.

Docker: VSCode

● Pull/push ...

● Previo logueo con usuario/pass de dockerhub



Docker: Construir y subir imagenes

● `docker build -t DOCKER_USERNAME/getting-started-todo-app .`

● <code>docker build</code>	Crea una imagen Docker a partir del archivo Dockerfile.
<code>-t</code>	Asigna una etiqueta (tag) con el formato usuario/nombre:tag.
<code>docker_user/getting-started-todo-app</code>	Nombre que tendrá la imagen.
<code>.</code>	Directorio actual, donde está el Dockerfile.

Docker leerá el Dockerfile.

Copiará los archivos del proyecto.

Instalará dependencias.

Y generará una imagen lista para ejecutar o subir a Docker Hub.

Docker: Construir y subir imagenes

● docker image ls

Lista de todas las imágenes almacenadas localmente en tu equipo.

REPOSITORY

TAG

IMAGE ID

CREATED

SIZE

Nombre o repositorio de la imagen.

Etiqueta o versión de la imagen (latest, v1, etc.).

Identificador único de la imagen.

Cuándo se creó la imagen.

Tamaño total de la imagen.

```
PS C:\Users\diego> docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
getting-started-todo-app-main-backend	latest	e5bc2fdc384a	About an hour ago	1.28GB
getting-started-todo-app-main-client	latest	537e502979ef	About an hour ago	1.25GB
diegoayaladocker/docker-quickstart	latest	1828a97621f9	6 days ago	141MB
diegoayaladocker/getting-started-todo-app	latest	53cbb5966952	13 days ago	1.21GB
<none>	<none>	fd775413e9fa	2 weeks ago	1.25GB
<none>	<none>	9166401c7c2f	2 weeks ago	1.24GB
docker/welcome-to-docker	latest	6caf772f5178	2 months ago	14.1MB
traefik	v3.4	aa215d9f973d	2 months ago	227MB
mysql	9.3	850100bac3be	5 months ago	859MB
phpmyadmin	latest	2d9a07a01a58	8 months ago	568MB

Docker: Construir y subir imagenes

- `docker push DOCKER_USERNAME/getting-started-todo-app`

Sube tu imagen local a Docker Hub.

Docker - Ejercicio 4

- Sube tu imagen a docker mediante comandos, llamándose ejercicio4.

Adjunta las capturas del proceso

Docker - Ejercicio 5

- Sube tu imagen a docker mediante vscode.

Adjunta el resultado final

Docker: El problema inicial

Escenario:

Imagina que estás desarrollando una *web app* con tres componentes:

-  **Frontend:** React
-  **Backend:** Django en Python
-  **Base de datos:** SQLite

Problema:

- Tienes que instalar Node, Python y SQLite en tu máquina.
- ¿cómo te aseguras de tener las **mismas versiones** que tus compañeros o el entorno de producción?

Docker: Los conflictos

- ¿Cómo garantizas que usas **las mismas versiones** de Node, Python o PostgreSQL que tu equipo?
- ¿Cómo evitas conflictos con lo que ya tienes instalado en tu máquina?
- ¿Cómo aseguras que todo funcione igual en **desarrollo**, **CI/CD** y **producción**?

Docker: Contenedores

Los contenedores permiten ejecutar cada componente en un entorno aislado.

- React → un contenedor
- Django Python → otro contenedor
- SQLite → otro contenedor

Cada uno es un **proceso independiente y aislado**, sin interferir con el resto.

Docker: Qué es un contenedor

Un contenedor es un **proceso aislado** que contiene **todo lo necesario** para ejecutar una aplicación.

Incluye:

- Código de la app
- Dependencias y librerías
- Variables de entorno
- Configuración

Todo en un solo paquete reproducible.

Docker: Ventajas principales

- **Autocontenidos:** no dependen de nada del sistema host.
- **Aislados:** mayor seguridad, sin interferencias entre sí.
- **Independientes:** puedes borrar uno sin afectar al resto.
- **Portables:** funcionan igual en cualquier entorno (local, nube o data center).

Docker: Contenedores vs Máquinas Virtuales

Máquina Virtual (VM):

- Contiene un **sistema operativo completo** (con su kernel, drivers, programas...).
- Mucho peso y sobrecarga si solo quieres ejecutar una app.

Contenedor:

- Es un **proceso aislado**, no un SO completo.
- Todos los contenedores **comparten el mismo kernel** del host.

Resultado: más eficiencia y menos consumo de recursos.

Docker: Ver contenedores

- docker ps → muestra contenedores en ejecución

```
PS C:\Users\diego> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
11db11cd73b6	docker/welcome-to-docker	"/docker-entrypoint..."	4 minutes ago	Up 4 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	quizzical_solomon
253bdc789745	nginx	"/docker-entrypoint..."	36 minutes ago	Up 36 minutes	0.0.0.0:80->80/tcp, [::]:80->80/tcp	wonderful_hypatia

- docker ps -a → muestra todos los contenedores

```
PS C:\Users\diego> docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
11db11cd73b6	docker/welcome-to-docker	"/docker-entrypoint..."	4 minutes ago	Up 4 minutes	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	quizzical_solomon
253bdc789745	nginx	"/docker-entrypoint..."	37 minutes ago	Up 37 minutes	0.0.0.0:80->80/tcp, [::]:80->80/tcp	wonderful_hypatia
39c10b1797ce	getting-started-todo-app-main-backend	"/docker-entrypoint.s..."	24 hours ago	Exited (255) 37 minutes ago		getting-started-todo-app-main-backend-1
0f5235c7346a	getting-started-todo-app-main-client	"/docker-entrypoint.s..."	24 hours ago	Exited (255) 37 minutes ago		getting-started-todo-app-main-client-1
c8c4985286a9	docker/welcome-to-docker	"/docker-entrypoint..."	24 hours ago	Exited (255) 37 minutes ago	0.0.0.0:8080->80/tcp	cranky_mclaren
2803f8206a3e	docker/welcome-to-docker	"/docker-entrypoint..."	7 days ago	Exited (255) 24 hours ago	0.0.0.0:8081->80/tcp	elastic_dubinsky
707f8622c52e	docker/welcome-to-docker	"/docker-entrypoint..."	7 days ago	Created		sharp_meninsky
6b47900a1614	docker/welcome-to-docker	"/docker-entrypoint..."	7 days ago	Exited (255) 24 hours ago	80/tcp, 0.0.0.0:9090->90/tcp	competent_bassi
412c6dceae01	docker/welcome-to-docker	"/docker-entrypoint..."	7 days ago	Created		zealous_lederberg
8affae507d67	diegoayaladocker/docker-quickstart	"/docker-entrypoint.s..."	7 days ago	Exited (255) 24 hours ago	0.0.0.0:8080->8080/tcp	upbeat_sinoussi
fc7e6295670e	traefik:v3.4	"/entrypoint.sh --pr..."	2 weeks ago	Exited (255) 37 minutes ago	0.0.0.0:80->80/tcp	getting-started-todo-app-main-proxy-1
eb24f15a231c	phpmyadmin	"/docker-entrypoint..."	2 weeks ago	Exited (255) 37 minutes ago	80/tcp	getting-started-todo-app-main-phpmyadmin

Docker: port host vs port contenedor

PC (host)

└─ puerto 8080 → Contenedor Docker
 └─ puerto 80 (app web)

- El contenedor ejecuta un servidor web en su **puerto 80**.
- No puedes acceder directamente al puerto 80 del contenedor.
- Al abrir <http://localhost:8080> Docker redirija esa conexión a contenedor:80

Docker: El problema

- Si un contenedor es un proceso aislado...
 - ¿de dónde obtiene sus archivos y configuración?
 - ¿cómo se comparten esos entornos entre diferentes equipos?

Imágenes de contenedor

Docker: ¿Qué es una imagen?

● **Imagen** → **Paquete estandarizado** que incluye todo lo necesario para ejecutar un contenedor:

- Archivos del sistema
- Binarios y librerías
- Configuraciones
- Dependencias

Plantilla para crear contenedores idénticos.

Docker: ¿Qué es una imagen?

● *Imagen de PostgreSQL →*

- Binarios de la base de datos
- Ficheros de configuración
- Dependencias necesarias

● *Imagen de una app Python →*

- El runtime de Python
- Tu código fuente
- Librerías y dependencias

Docker images: Inmutabilidad

● Las imágenes son inmutables

- Una vez creada, **no se puede modificar**.
- Solo puedes:
 - Crear una nueva versión
 - O **añadir capas encima** (nuevas modificaciones).

Esto garantiza **consistencia y reproducibilidad** en todos los entornos.

Docker images: Capas

● Las imágenes están formadas por capas (layers):

- Cada capa representa un cambio en el sistema de archivos.
- Puede **añadir, eliminar o modificar** archivos.

Capa 1 → Base de Ubuntu

Capa 2 → Instala Python

Capa 3 → Añade tu aplicación

Capa 4 → Configuración final

Docker images: Crear tus propias imágenes

● Puedes **extender imágenes existentes**:

Por ejemplo:

- Partes de python:3.11
- Añades tus dependencias (pip install ...)
- Copias tu código y configuración

Resultado → tu propia imagen personalizada.

Te enfocas en **tu aplicación**, no en reinstalar entornos (VBox, SSII).

Docker: dockerfile

- Dockerfile

```
FROM python:3.11
WORKDIR /app
COPY . /app
RUN pip install --no-cache-dir PySide6
EXPOSE 5000
CMD ["python", "main.py"]
```

Docker images: DockerHub

- Buscar imágenes
- Descargarlas
- Ejecutarlas directamente desde Docker Desktop

¿Con qué ejemplo predefinido hemos trabajado de DockerHub?

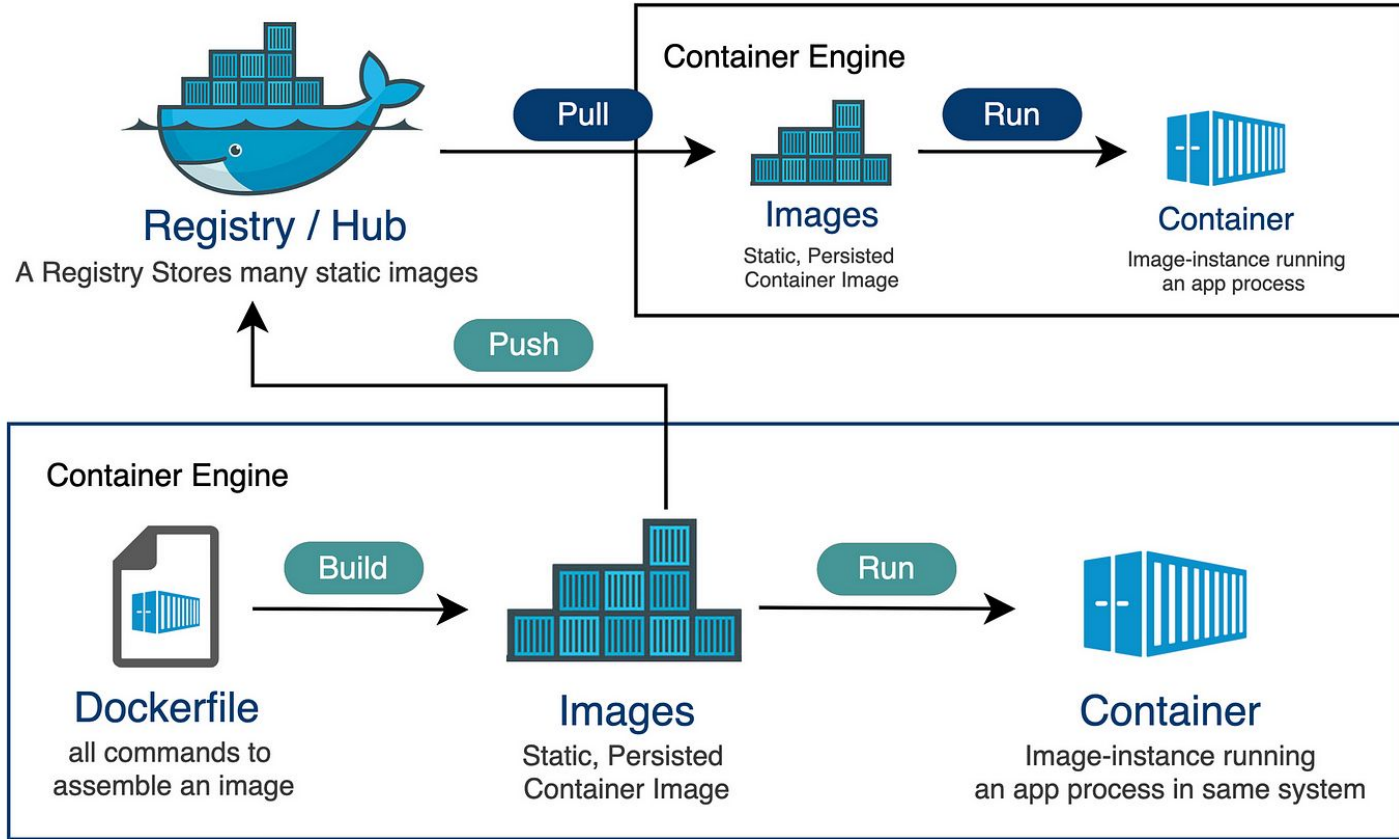
Docker: Registry

- Servidor o servicio centralizado donde se almacenan y comparten los repositorios.

→ Docker Hub, Amazon ECR, GitLab Registry

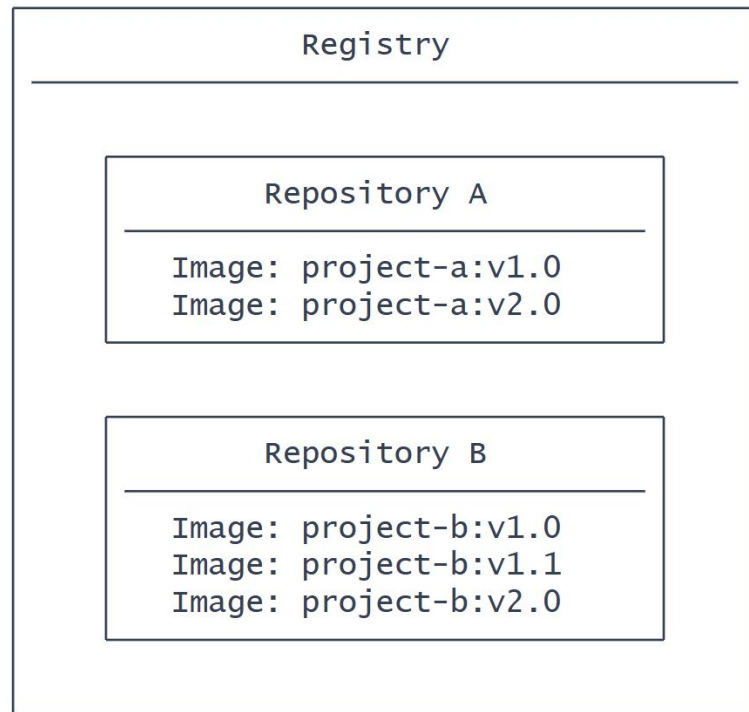


Docker images: Ciclo de vida



Docker: Registry vs. repository

- Repository → Colección de imágenes relacionadas dentro de un registro
→ usuario/miapp con versiones v1, v2



Docker Compose

- Nuestra app se encuentra **en un contenedor**.
¿si queremos añadir más servicios?

Bases de datos
Colas de mensajes
Cachés

- Ejecutar múltiples contenedores → ¿Como los conectamos?
→ Múltiples docker run que deben comunicarse
entre sí

Docker - DockerFile

- DockerFile → Contiene las instrucciones para construir una imagen.

```
FROM python:3.13
WORKDIR /usr/local/app
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
COPY src ./src
EXPOSE 8080
RUN useradd app
USER app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Docker - DockerFile

- 1 Elegir imagen base (FROM)
- 2 Definir directorio de trabajo (WORKDIR)
- 3 Copiar dependencias y código (COPY)
- 4 Instalar librerías (RUN)
- 5 Exponer puerto (EXPOSE)
- 6 Crear usuario no root (USER)
- 7 Definir comando de inicio (CMD)

Docker- Contenedores one-off

- Ejecutan una tarea puntual y terminan.
- No es interactivo ni hay procesos a la espera
- Se inician con docker run y finalizan cuando acaba el proceso.

Características:

- Vida corta
- Reproducibles y desechables
- Útiles en CI/CD

```
PS C:\Users\diego> docker run alpine hostname  
065f3d0e9e44
```

Docker- Contenedores de larga duracion

- Ejecutan procesos que permanecen activos (p. ej., servicios)

Nginx, bases de datos, APIs

- Se puede conectar el terminal al proceso usando -it para administrarlos

-i stdin -t stdout terminal format

Docker - Ejercicio 6

- Convierte el contenedor alpine de one-off a uno de larga duración en el cual puedas ejecutar comandos.

Docker- Entorno aislado

- Lo que escribes en tu terminal se ejecuta dentro del contenedor
- Aísla procesos, FS y red del host
- Ideal para pruebas controladas y entornos limpios

```
PS C:\Users\diego> docker run -it ubuntu  
root@31bf8b961997:/# curl marca.com  
bash: curl: command not found  
root@31bf8b961997:/# ping 8.8.8.8  
bash: ping: command not found  
root@31bf8b961997:/# |
```

Docker - Ejercicio 9

Hemos ejecutado contenedores usando **Alpine Linux** y **Ubuntu**, y también con **Nginx** instalado. Todos ellos son **paquetes públicos disponibles en Docker Hub**.

En este laboratorio trabajarás con **contenedores de Java**:

1. Busca en **Docker Hub** una imagen que puedas usar para ejecutar una aplicación Java.

Ejecuta un contenedor de Java y confirma **qué versión de Java** está instalada usando el comando:

2. `java -version`
3. Ahora busca una **imagen más pequeña de Java**, que solo incluya el **entorno de ejecución JRE (Java Runtime Environment)**.

Docker - Ejercicio 10

1. 10

Inicio de la web - Peticiones HTTP

- La Web comenzó como un sistema muy simple.
- Los servidores **solo podían devolver archivos estáticos.**

- **Definición**

Tipo de contenido

Sin lógica

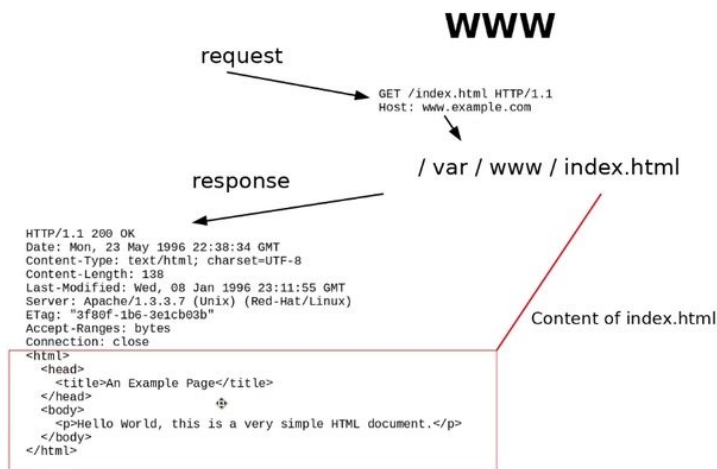
Simplicidad

Implementación real

Solo archivos estáticos (HTML, CSS, imágenes...)

No podía ejecutar código dinámico

El servidor solo leía y enviaba archivos



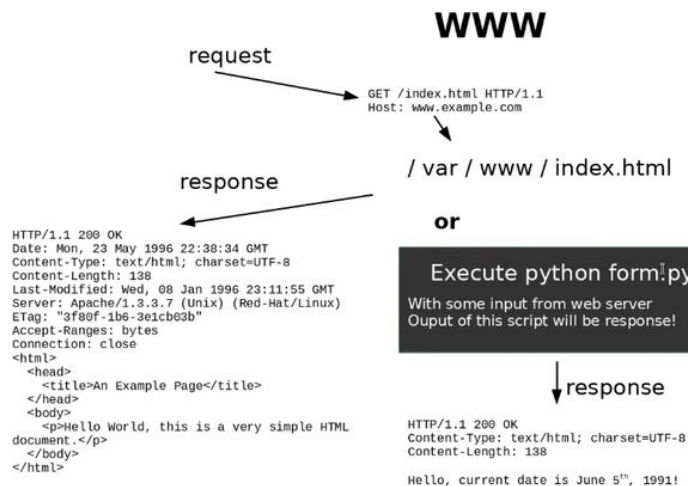
Web dinámica

- Empezaron a aparecer formularios y necesidad de procesar datos.

→ Responder dinámicamente según lo que envíe el usuario.

- Procesar formularios (login, registro...)
- Guardar datos de usuario (sesiones)
- Generar páginas diferentes según la entrada

- Los servidores debían ejecutar código, no solo devolver archivos.



CGI

- Cada servidor web pasaba los datos a los scripts de forma distinta (variables de entorno distintas)
- CGI → Estándar servidor web debe pasar información a los scripts
- Que cualquier script (Python, Perl, PHP, etc.) pueda ejecutarse igual en cualquier servidor.



CGI

- Cada servidor web (Apache, NGinx ...) pasaba la información al script a su manera:
 - Uno llamaba a la variable `SERVER_NAME`
 - Otro la llamaba `HOSTNAME`
 - Otro directamente no la pasaba
- Los scripts no eran portables.
- Si cambiabas de servidor, tu código dejaba de funcionar.

CGI

- Cuando llega una petición HTTP:

1. El servidor web la recibe
2. La convierte en variables de entorno
3. Ejecuta el script (por ejemplo form.py)
4. El script lee esas variables y genera la respuesta
5. El servidor envía esa salida al navegador

- CGI es el puente entre el servidor web y el programa que genera la respuesta.

`SERVER_SOFTWARE`: name/version of HTTP server

`SERVER_NAME`: host name of the server, may be dot-decimal IP address.

`SERVER_PROTOCOL`: HTTP/version.

`SERVER_PORT`: TCP port (decimal).

`REQUEST_METHOD`: name of HTTP method

`PATH_INFO`: path suffix, if appended to URL after program name and a slash.

`SCRIPT_NAME`: relative path to the program, like /cgi-bin/script.cgi.

`QUERY_STRING`: the part of URL after ? character.

`REMOTE_HOST`: host name of the client, unset if server did not perform such lookup.

`REMOTE_ADDR`: IP Address of the client (dot-decimal).

WSGI (Web Server Gateway Interface)

- Estándar creado en 2003
- Define cómo un servidor web debe comunicarse con aplicaciones Python.
- CGI estandarizó las variables
- WSGI estandariza cómo ejecutar código Python para servir páginas web.

¿Por qué era necesario WSGI?

● Antes:

- Los scripts Python se ejecutaban desde cero en cada petición (CGI).
- Cada servidor lo hacía de formas diferentes.
- Era lento y difícil de mantener.

● Con WSGI:

- La aplicación Python se mantiene en memoria.
- El servidor solo pasa la petición al código.
- Mucho más rápido y portátil.

WSGI en código

- ```
def app(env, start_response):
 status = "200 OK"
 headers = [("Content-Type", "text/plain")]
 start_response(status, headers)
 return [b"Hola desde WSGI"]
```
- env → diccionario con valores CGI (REQUEST\_METHOD, PATH\_INFO...)
- start\_response → función para enviar estado + cabeceras
- Se devuelve el cuerpo de la respuesta

→ Toda app Python compatible con WSGI funciona igual.

# Descomponiendo WSGI

## ● Parte

Aplicación / Framework (Flask, Django, FastAPI\*)

Servidor WSGI (Gunicorn, uWSGI, mod\_wsgi)

## Rol

Genera la respuesta

Ejecuta callable y comunica con servidor web

● \* FastAPI usa ASGI, evolución de WSGI (asíncrono).

# ¿Por qué no Apache o Nginx directamente?

- Apache y Nginx NO ejecutan Python (ni entienden cómo llamar a un app WSGI)
- Necesitamos un servidor WSGI que haga de puente:

Navegador → Nginx/Apache → Gunicorn/uWSGI → Tu App WSGI (Flask/Django)

# Ventajas de los Servidores WSGI

## ● Ventaja

No reinicia Python cada vez

Pre-forking (workers)

Escalable

Compatible

## Explicación

Código permanece en memoria → Más rápido

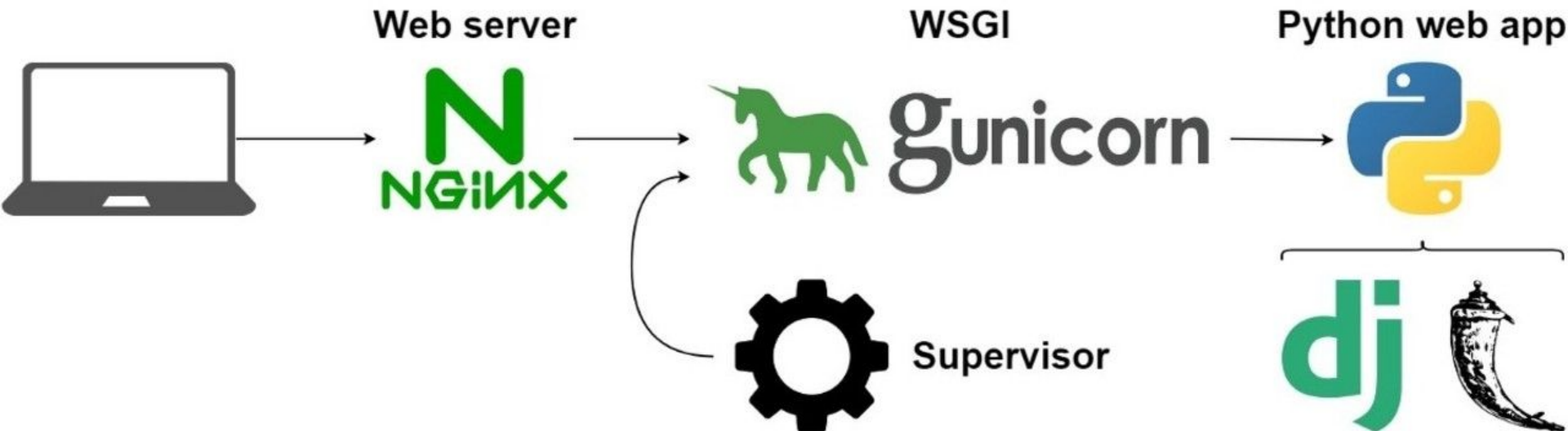
Procesos listos para atender peticiones → Mejor rendimiento

Se puede ajustar número de workers según carga

Funciona igual en cualquier servidor web

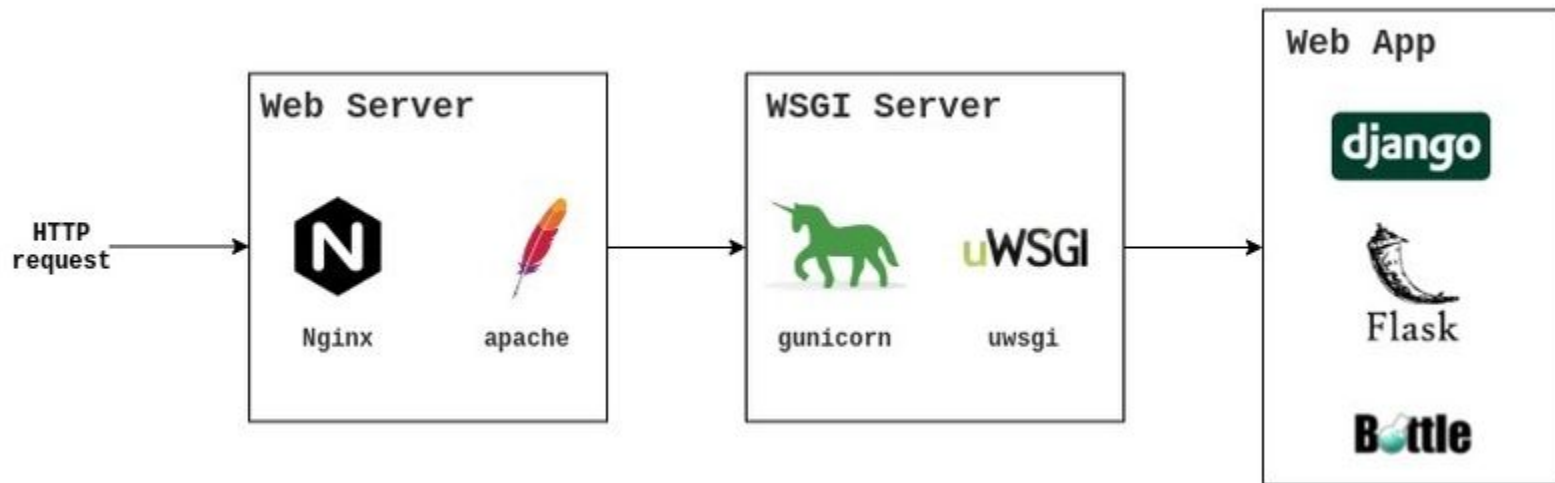
# WSGI (Web Server Gateway Interface)

- WSGI hizo posible que las aplicaciones Python fueran rápidas, portables y compatibles entre servidores.
- Es la base sobre la que funcionan Flask y Django en producción.



# ¿Qué es un Servidor Web?

- Recibe las peticiones HTTP del navegador.
- Puede servir archivos estáticos.
- Puede redirigir peticiones a otra aplicación.
- Es la “puerta de entrada” del sistema.



# Web Server (Nginx / Apache)

- El servidor web es el que:
- Recibe la petición HTTP del navegador.
- Se encarga de cosas de red: puertos, HTTPS, certificados, archivos estáticos (CSS/JS/Imágenes).
- Pero NO sabe ejecutar código Python.
- Por eso no puede hablar directamente con Flask o Django.

→ Necesita un intermediario .



**NGINX**®  
Part of F5



THE  
**APACHE**™  
SOFTWARE FOUNDATION

# WSGI Server (Gunicorn / uWSGI)

- Convertir la petición en algo que Python entienda.
- Ejecutar tu aplicación.
- Devolver la respuesta al servidor web.

Mantiene el intérprete Python en memoria.

Puede crear workers para manejar más peticiones (escala).



# Web App (Flask / Django)

- Aquí está tu código Python:
  - Define rutas.
  - Procesa formularios.
  - Genera HTML o JSON.
  - Usa bases de datos, sesiones, etc.
- La aplicación no atiende directamente peticiones HTTP, porque no habla HTTP.
- Habla WSGI.

# De 1 contenedor → 2 contenedores comunicándose

● Antes:

**Navegador → (puerto expuesto) → Flask (servidor de desarrollo)**

● Funciona pero NO hemos desplegado en producción.

● Problemas:

- Flask trae un servidor de desarrollo, no optimizado (sin workers)
- La app queda expuesta directamente a Internet.
- No hay control de tráfico, seguridad, ni logging frontal.

# Despliegue en prod

- Nueva arquitectura en prod:

**Navegador → Nginx (Reverse Proxy) → Gunicorn → Flask**

- Nginx recibe las peticiones externas.
- Gunicorn ejecuta Flask en modo producción.
- Flask se centra solo en la lógica.



**NGINX**®  
Part of F5



THE  
**APACHE**™  
SOFTWARE FOUNDATION

# ¿Por qué 2 contenedores?

| ● Contenedor     | Rol                               | Por qué separado    |
|------------------|-----------------------------------|---------------------|
| Flask + Gunicorn | Ejecuta la app                    | Escala según carga  |
| Nginx            | Entrada pública, proxy, seguridad | Se configura aparte |

1 contenedor = 1 proceso / 1 responsabilidad

# Docker Network

- Para que dos contenedores se comuniquen, deben estar en la misma red Docker:

**`docker network create webnet`**

- Después:

**`docker run --network webnet ...`**

→ Dentro de esa red, los contenedores se pueden llamar por su nombre.

→ Equivalente a una LAN privada solo entre contenedores.

# ¿Qué es Nginx?

- Servidor web **muy rápido**
- Hace de **reverse proxy**
- Decide **a qué servicio** pasa una petición

Puede aplicar:

- cabeceras de seguridad
- compresión gzip
- reglas de filtrado
- *rate limiting* (más adelante)

# proxy\_pass

- **location / {**

```
proxy_pass http://fiesta_app:8000;
```

```
proxy_set_header Host $host;
```

```
proxy_set_header X-Real-IP $remote_addr;
```

```
}
```

Cuando llega una petición a /, reenvíala al contenedor fiesta\_app en su puerto interno 8000.

[Documentación Oficial](#)

# Nginx – Publicar el servicio

- Todas las peticiones que llegan a la raíz del servidor (/) serán manejadas por este bloque.

- Ejemplos:

/

/health

/loquesea

# Nginx – location / { ... }

● proxy\_pass [http://fiesta\\_app:8000](http://fiesta_app:8000);

→ Nginx pasa la petición al contenedor donde está Flask.

fiesta\_app → nombre del contenedor que corre Unicorn + Flask.

8000 → es el puerto interno donde Unicorn está escuchando dentro de la red Docker.

```
location / {

 proxy_pass http://fiesta_app:8000;

 proxy_set_header Host $host;

 proxy_set_header X-Real-IP $remote_addr;

}
```

# Nginx – location / { ... }

- `proxy_set_header Host $host;`

→ Sobrescribe la cabecera Host para que la app sepa a qué dominio se hizo la petición.

Si el usuario entra desde: `http://localhost:8080`

La app verá:

Host: localhost

```
location / {

 proxy_pass http://fiesta_app:8000;

 proxy_set_header Host $host;

 proxy_set_header X-Real-IP $remote_addr;

}
```

# Nginx – location / { ... }

● `proxy_set_header X-Real-IP $remote_addr;`

- Por defecto, Flask solo ve la IP de Nginx, no la del navegador.

Esto corrige eso:

Le pasa la IP real del cliente para:

- El cliente tiene IP: 192.168.1.45
- Nginx envía:
- X-Real-IP: 192.168.1.45

```
location / {

 proxy_pass http://fiesta_app:8000;

 proxy_set_header Host $host;

 proxy_set_header X-Real-IP $remote_addr;

}
```

# Publicar el servicio

- Flask no se publica.
- Gunicorn escucha solo dentro de la red.
- El único servicio público es Nginx:

```
docker run -p 8080:80 nginx_proxy
```

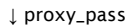
→Accedemos desde el navegador:

<http://localhost:8080>

(Internet/Navegador)



Nginx (puerto 8080 en el host)



Gunicorn (puerto 8000 interno)



Flask

# Publicar un servicio dedicado

- `docker run -d --name nginx_proxy --network webnet -p 8080:80`  
`-v ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro nginx:1.27-alpine`

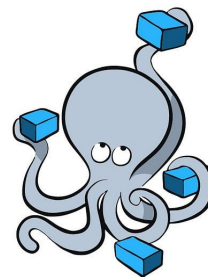


# DAPPW - 2.Docker Compose

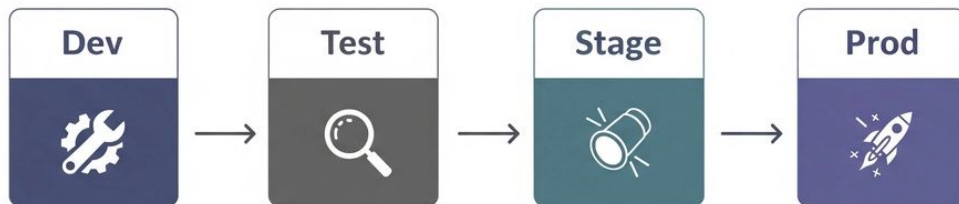
Diego Ayala

# ¿Qué es Docker Compose?

- Herramienta para definir y ejecutar **aplicaciones multi-contenedor**.
- Orquesta servicios, redes y volúmenes con **un único archivo compose.yaml**.
- Un solo comando para levantar todo el stack.
- Útil en **desarrollo, testing, staging, producción y CI/CD**.
- Puente hacia otras plataformas (**K8s**)



**docker**  
Compose



# El sueño de los zánganos

Define your app's  
environment with a  
Dockerfile

Define the services that  
make up your app in  
Docker Compose file

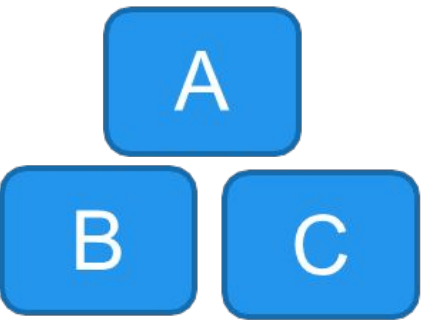
Run the CLI:

*\$ docker-compose up*

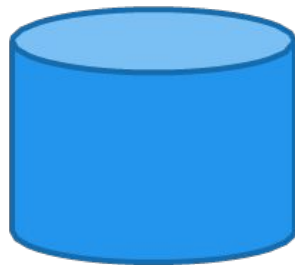


# ¿Qué es el archivo Compose?

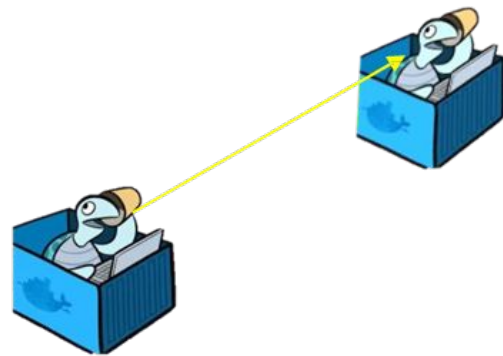
- Archivo YAML (compose.yaml) → define los servicios, redes, volúmenes, secretos y configs de la aplicación.
- Sigue la especificación de Compose Specification.
- Aplicaciones compuestas por varios contenedores que interactúan.



Services



Volumes



Networking

# compose.yaml



## Elemento

## Como interactua

## Que crea

services

Contenedores que forman la app

Contenedores

networks

Redes entre servicios

Conexión automática entre contenedores

volumes

Almacenamiento persistente

Aunque el contenedor muera

secrets

Datos sensibles (contraseñas, tokens)  
gestionados como ficheros de solo lectura

Archivos montados (  
/run/secrets/...) accesibles solo por el  
servicio.

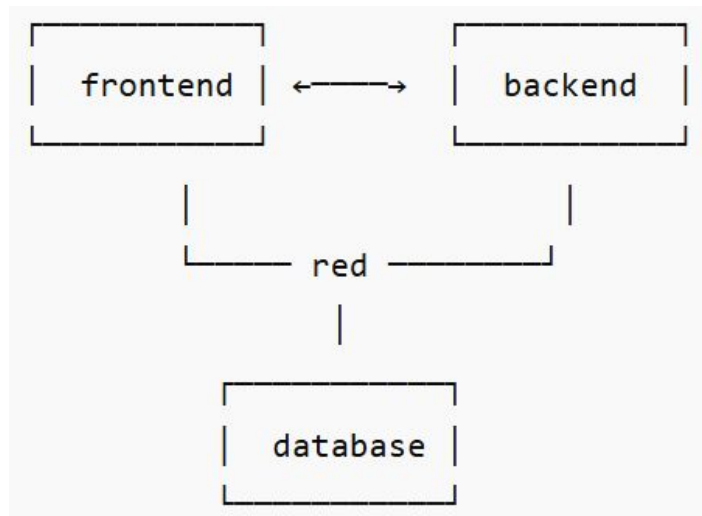
configs

Archivos de configuración no sensibles  
(nginx.conf, app.ini) de solo lectura

Archivos montados en el contenedor para  
su lectura por la app

# compose.yaml

- Todos los contenedores comunican mediante nombres.
- No hace falta usar IPs o puertos (solo para la comunicación dentro del contenedor)



# compose.yaml – Servicios

- Un servicio en Compose es un contenedor con configuraciones predeterminadas.
- Ejecuta el Dockerfile de ./backend → **build vs image**

**Toma el Dockerfile de ./backend**

**Arranca siempre con la variable DEBUG activada**

```
services:
 ▷ Run Service
 api:
 build: ./backend
 environment:
 - DEBUG=true
```

# compose.yaml

- web usa una imagen nginx
- Se expone el puerto 8080 al host
- db inicia PostgreSQL con contraseña
- Ambos se conectan automáticamente a mi\_red

```
services:
 ▷ Run Service
 web:
 image: nginx:alpine
 ports:
 - "8080:80"

 ▷ Run Service
 db:
 image: postgres:16
 environment:
 POSTGRES_PASSWORD: ejemplo

networks:
 default:
 name: mi_red
```

# compose.yaml – Redes

- Compose crea una red interna privada.
  - Los contenedores se reconocen por nombre
  - Seguridad → la red no es accesible desde fuera
  - Facilita la comunicación backend ↔ database



```
services:
 ▷ Run Service
 web:
 image: nginx:alpine
 ports:
 - "8080:80"

 ▷ Run Service
 db:
 image: postgres:16
 environment:
 POSTGRES_PASSWORD: ejemplo

networks:
 default:
 name: mi_red
```

# compose.yaml – Acceso desde localhost

- `http://web` NO funciona desde tu PC, porque tu PC no está dentro de la red interna.
- Para acceder desde fuera, usamos el puerto publicado:

services:

web:

ports:

- "8080:80" → PUERTO\_DEL\_HOST : PUERTO\_DEL\_CONTENEDOR

`http://localhost:8080` ✓

`http://web` ✗



```
services:
 web:
 image: nginx:alpine
 ports:
 - "8080:80"

 db:
 image: postgres:16
 environment:
 POSTGRES_PASSWORD: ejemplo

networks:
 default:
 name: mi_red
```

# compose.yaml – Volúmenes

- Sirven para guardar datos aunque el contenedor se borre.
- Sin volumen → la base de datos se borra al reiniciar el contenedor.

```
services:
 db:
 image: postgres:16
 environment:
 POSTGRES_PASSWORD: pass
 volumes:
 - datos_db:/var/lib/postgresql/data

volumes:
 datos_db:
```

# compose.yaml – Configs y Secrets

- Sirven para guardar datos aunque el contenedor se borre.
- Sin volumen → la base de datos se borra al reiniciar el contenedor.

```
services:
 ▷ Run Service
 web:
 image: httpd:2.4-alpine
 ports:
 - "8080:80"
 configs:
 - source: apache_conf
 target: /usr/local/apache2/conf/httpd.conf
 volumes:
 - ./www:/usr/local/apache2/htdocs:ro
 secrets:
 - api_key
 ▷ Run Service
 db:
 image: postgres:16
 environment:
 POSTGRES_DB: ejemplo
 POSTGRES_USER: usuario
 POSTGRES_PASSWORD_FILE: /run/secrets/pg_pass
 secrets:
 - pg_pass
```

```
configs:
 apache_conf:
 file: ./apache/httpd.conf
```

```
secrets:
 api_key:
 file: ./secrets/API_KEY.txt
 pg_pass:
 file: ./secrets/POSTGRES_PASSWORD.txt
```

# compose.yaml – Comandos



## Comando

`docker compose up -d (--build)`

`docker compose ps`

`docker compose logs -f`

`docker compose down`

## Acción

Levanta la app

Muestra servicios corriendo

Ver logs en tiempo real

Detiene y borra contenedores

(-v resetea también los volúmenes)

# Docker Compose - Ejercicio 1

● <https://drive.google.com/file/d/1I9znLac-E7TxhHEAtZE3bZedqWWGcCJf/view?usp=sharing>

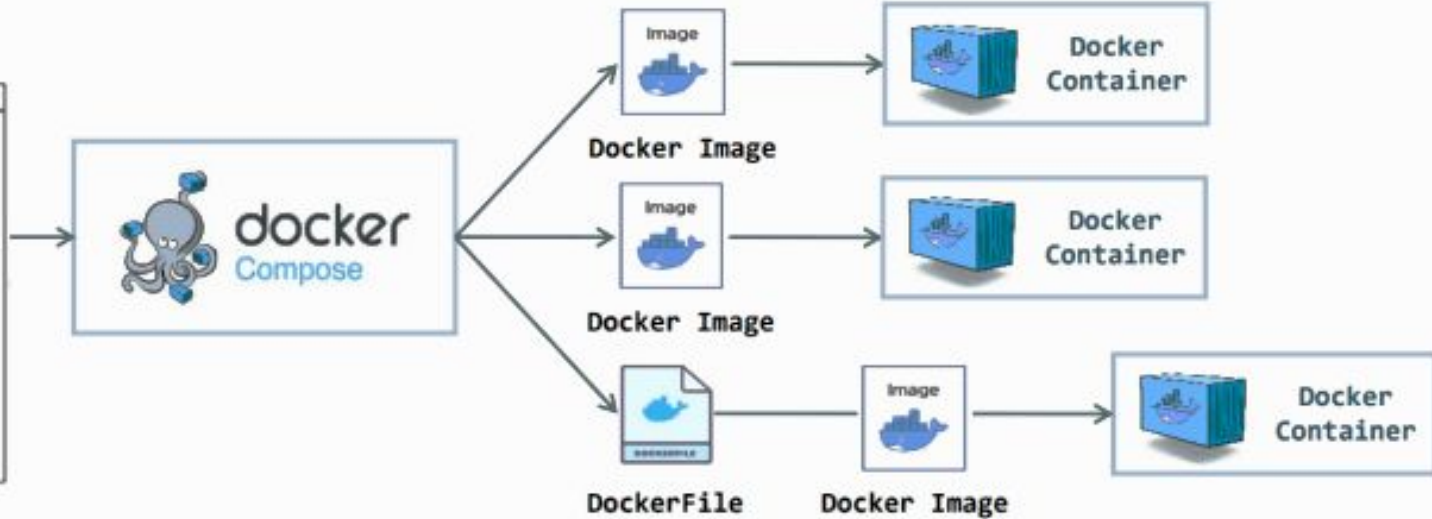
# Docker Compose - Ejercicio 2

- Dockeriza con compose el ejercicio de flask-cat-app

# Docker Compose - Visión general

docker-compose.yml

```
*** docker-compose.yml ***
version: "3.7"
services:
 db:
 image: mysql:5.0.19
 restart: always
 environment:
 - MYSQL_DATABASE=example
 - MYSQL_ROOT_PASSWORD=password
 app:
 build: app
 restart: always
 web:
 build: web
 restart: always
 ports:
 - 8080
```



# Compose – Sustitución de variables de entorno

- `${VAR}` → sustituye VAR si existe; si no, queda en blanco.
- `${VAR:-valor}` → si VAR está definida y no vacía, usa su contenido; de lo contrario, usa valor.

```
environment:
- TEST=${TEST:-12}
- N_ZANGANOS=${NZANGANOS:-20}
```

¿De donde cargan sus valores? → Mejor praxis .env

# Compose – .env

- Archivo de texto plano con pares clave-valor de variables de entorno.
- Define valores de configuración → no los escribes en el código o el docker-compose.yml

⚙️ .env

```
1 TEST=4
2 N_ZANGANOS=21
3 APP_PORT=8080
4 APP_ENV=development
```

environment:

```
- TEST=${TEST:-12}
- N_ZANGANOS=${NZANGANOS:-20}
```

# Docker Compose - Ejercicio 3

- Dockeriza con compose el ejercicio de dice-app. Deberás usar un archivo .env para poder cambiar el número de caras de los dados y por defecto si no existe dicho archivo env o estuviera vacío el valor por defecto debe ser 6

# Compose – Type Schema

- Cada clave del compose tiene un tipo esperado.
- Docker Compose valida ese tipo al interpretar el archivo.

Scalar

image: nginx:alpine

Mapping

build: { context, dockerfile }

Array (lista)

ports: - "8080:80"

# Compose – Scalar

- Cada clave del compose tiene un tipo esperado.
- Docker Compose valida ese tipo al interpretar el archivo.

Scalar

`image: nginx:alpine`

Mapping

`build: { context, dockerfile }`

Array (lista)

`ports: - "8080:80"`

# Compose – Mapping

- Cada clave del compose tiene un tipo esperado.
- Docker Compose valida ese tipo al interpretar el archivo.

Scalar

`image: nginx:alpine`

Mapping

`build: { context, dockerfile }`

Array (lista)

`ports: - "8080:80"`

# Compose – Array

- Cada clave del compose tiene un tipo esperado.
- Docker Compose valida ese tipo al interpretar el archivo.

Scalar

image: nginx:alpine

Mapping

build: { context, dockerfile }

Array (lista)

ports: - "8080:80"

# LAURA FELIZ



# LAURA TRISTE



# Entornos virtuales – venv

● Despliegue mínimo en local:

**-Declarando env:**

```
py -m venv .venv
```

**-Entrando env:**

```
.venv\Scripts\activate
```

**-Actualizar pip (buena praxis):**

```
py -m pip install --upgrade pip || pipenv lock --requirements
```

**-Instalando dependencias:**

```
pip install "django==5.2.6" django-tailwind
"django-tailwind[reload]" pillow
```

**-Cambios en el ORM:**

```
py manage.py makemigrations
```

**-Aplicamos cambios ORM:**

```
py manage.py migrate
```

**-Ejecutamos server:**

```
py manage.py runserver
```

# requirements.txt– Versionarlo

- pip freeze → Lista los paquetes instalados junto con sus versiones

```
Django==5.2.6
test==4.3.1
test3|=12.0.0
```

```
Flask>=3.0,<4
```

```
PS C:\Users\diego> pip freeze
altair==5.5.0
asgiref==3.10.0
asn1crypto==1.5.1
attrs==25.4.0
blinker==1.9.0
cachetools==6.2.0
certifi==2025.10.5
cffi==2.0.0
charset-normalizer==3.4.3
click==8.3.0
colorama==0.4.6
coverage==7.10.7
cryptography==46.0.2
Django==5.2.7
django-browser-reload==1.21.0
django-tailwind==4.3.1
Flask==3.1.2
gitdb==4.0.12
GitPython==3.1.45
google-api-core==2.25.2
google-api-python-client==2.184.0
google-auth==2.41.1
```

# Entornos virtuales – pipenv

● Despliegue mínimo en local:

- Declarando env:** `pipenv install "django==5.2.6" django-tailwind "django-tailwind[reload]" pillow`
- Entrando env:** `pipenv shell`
- Actualizar pip (buena praxis):** `py -m pip install --upgrade pip || pipenv lock --requirements`
- Instalando dependencias:** `pipenv install`
- Cambios en el ORM:** `pipenv manage.py makemigrations`
- Aplicamos cambios ORM:** `pipenv manage.py migrate`
- Ejecutamos server:** `pipenv manage.py runserver`

# requirements.txt– Versionarlo

- pipenv requirements → Lista los paquetes instalados junto con sus versiones

```
-i https://pypi.org/simple
asgiref==3.10.0; python_version >= '3.9'
django==5.2.6; python_version >= '3.10'
django-browser-reload==1.21.0; python_version >= '3.9'
django-tailwind[reload]==4.4.1; python_version >= '3.11'
pillow==12.0.0; python_version >= '3.10'
pytailwindcss==0.3.0; python_version >= '3.11' and python_version < '4.0'
sqlparse==0.5.3; python_version >= '3.8'
tzdata==2025.2; python_version >= '2'
```

```
[[source]]
url = "https://pypi.org/simple"
verify_ssl = true
name = "pypi"

[packages]
django = "==5.2.6"
django-tailwind = {extras = ["reload"], version = "*" }
pillow = "*"

[dev-packages]

[requires]
python_version = "3.13"
```

# Compose – command

- command → Reemplaza el CMD del dockerfile.
  - sh -c: Ejecuta un shell con múltiples opciones.
  - Evalúa el bloque completo como solo una cadena.

```
command: >
| sh -c "python manage.py migrate --noinput &&
| | | | python manage.py runserver 0.0.0.0:8000"
```

# Compose – restart

- restart → El contenedor se reinicia automáticamente si se cae.
  - Exceptuando paradas manuales (docker compose down || stop)

```
restart: unless-stopped
```

# Compose – dev deploy

- `python manage.py runserver` → Pensado para desarrollar **SOLO** localmente
  - Recarga automática
  - **Sirve ficheros estáticos**
  - django browser reload para recarga automática ante cambios.

## CONTRAS:

- solo para desarrollo
- servidor minimalista e inseguro
- no sirve para alta carga ni múltiples procesos.

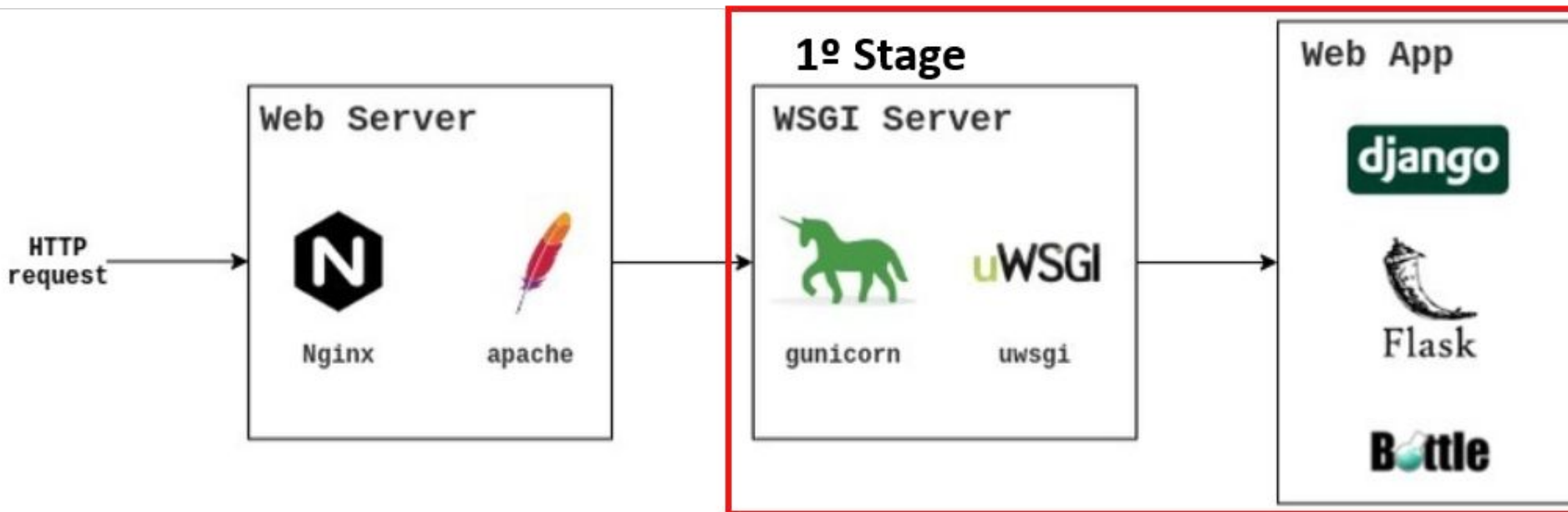
# Compose – prod deploy 1ºstage

Usamos gunicorn → `gunicorn modulo.archivo-config-wsgi:application --bind 0.0.0.0:8000`

wsgi → creado automáticamente por django

0.0.0.0 → contenedor escucha en todas las interfaces

:8000 → sirve en el puerto 8000



# Compose – prod deploy 1ºstage

- Problemas → django-browser-reload: Recarga dinámica de cambios en dev → mala praxis en prod
  - Envía eventos cada vez que detecta cambios en los archivos para recargarlos
  - Ineficiente

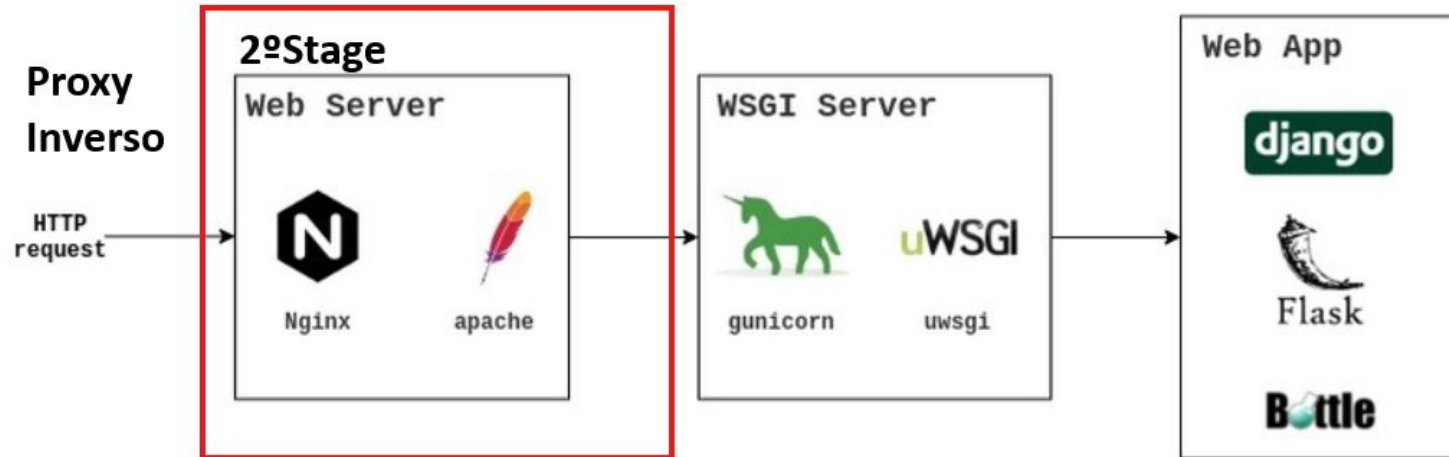
Deberemos eliminar todo rastro de reload: → requirements.txt && /modulo\_wsgi (todas las clases)

/personalblog

**django\_browser\_reload**

# Compose – prod deploy 2ºstage

- +Problemas → gunicorn NO es capaz de servir ficheros estáticos.
  - CSS, Tailwind: No disponible.
- Deberemos servirlos con un proxy inverso:



# Compose – prod deploy 2ºstage

- Configuración de nginx en local: ./nginx/conf.d/default.conf

/static/ → No proxy

/media/ → No proxy

location / → proxy a la dirección interna  
http://web:8000

```
server {
 listen 80;
 server_name _;

 location /static/ {
 alias /usr/src/app/staticfiles/;
 }

 location /media/ {
 alias /usr/src/app/media/;
 }

 location / {
 proxy_pass http://web:8000;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
 proxy_set_header X-Forwarded-Proto $scheme;
 }
}
```

# Compose – prod deploy 2ºstage

- Django y Tailwind → Dependencias estáticas repartidas entre múltiples directorios

# Compose – prod deploy 2ºstage

- settings.py → `STATIC_ROOT = BASE_DIR / "staticfiles"`

Define la carpeta donde Django volcará todos los archivos estáticos cuando ejecutes `python manage.py collectstatic`

# Compose – prod deploy 2ºstage

- Volúmenes: Volumen global compartido entre 2 contenedores y montado en distintas rutas.

/app(La raíz definida en el dockerfile)/staticfiles → por STATIC\_ROOT

staticfiles:/usr/src/app/staticfiles → Nginx lee los archivos que Django generó con collectstatic en el otro contenedor.

```
volumes:
 - staticfiles:/app/staticfiles
expose:
```

```
volumes:
 - ./nginx/conf.d:/etc/nginx/conf.d
 - staticfiles:/usr/src/app/staticfiles
 - ./media:/usr/src/app/media
```

```
volumes:
 staticfiles:
```